

ADH LLD Specialist Agent

1. Purpose & Scope

The LLD Specialist produces a **zero-ambiguity implementation specification** for coding agents. This is Phase 03 in the ADH SDLC pipeline. This will transform the approved high-level design into a complete technical blueprint that coding agents uses to develop the code.

This spec tells a developer exactly how to build `.github/agents/adh-lld.agent.md`. It does NOT cover the master orchestration, validation skills, or other specialists - those are separate specs.

Key principle: LLD must have complete implementation details.

2. Input → Output Contract

2.1 Inputs

Input	Source	What it provides
Delegation prompt	Master via <code>build_prompt.py</code>	Custodian config, project tracker state, all resolved paths
<code>requirements.md</code>	<code>.adh/workspace/output/{custodian}/ {project}/requirements.md</code>	Business requirements, approved in Phase 01
<code>hld.md</code>	<code>.adh/workspace/output/{custodian}/ {project}/hld.md</code>	High-level architecture, approved in Phase 02
<code>session-context.json</code>	<code>.adh/tracker/{custodian}/{project}/ session-context.json</code>	Key decisions from prior phases
ADH platform standards	Extracted from <code>.adh/standards/docs/</code> by <code>build_prompt.py</code>	Naming conventions (b_/s_/g_ prefixes), catalog layout, cluster defaults,

		ingestion_config schema (28 fields)
--	--	--

GUARD: Phase 03 CANNOT start unless `hld.md` status is `APPROVED` in tracker. Master enforces this.

2.2 Output

One file: `.adh/workspace/output/{custodian}/{project}/l1d.md`

Structure: 10 sections following the canonical template at `.adh/standards/artifacts/l1d.template.md`

Section	What it specifies
1. ingestion_config.json	All 28 fields - no field left unset or ambiguous
2. Bronze Layer DDL	Full table definition with <code>b_</code> prefix for all columns
3. Standard Layer DDL	Full table definition with <code>std_</code> prefix
4. Silver/Gold Layer DDL	Full table definitions (if applicable) with <code>s_</code> / <code>g_</code> prefixes
5. Job Configuration	Cluster spec, task chain, retry logic, timeout, run-as service principal
6. Notebook Specifications	Path, widgets, logic summary for each notebook
7. dbxmetagen Configuration	PII classification config (if PII fields present)
8. Data Quality Rules	Per-column validation checks, thresholds, actions
9. Validation Checklist	Pre-populated compliance checklist (used by Phase 04-06)
10. Key Decisions	ALL values coding phases will read from <code>session-context.json</code>

Template source: `.adh/standards/artifacts/lld.template.md` (canonical - do NOT embed a copy in agent instructions)

3. Deliverable

One file: `.github/agents/adh-lld.agent.md`

3.1 Frontmatter

Frontmatter = the YAML metadata block between `---` delimiters at the top of the `.agent.md` file. VS Code uses this to configure the agent's behavior, model, and tool permissions.

```
1 ---
2 name: "ADH LLD"
3 description: >
4   Phase 03 - Produces Low-Level Design document. Zero-ambiguity
5   implementation spec
6   for coding agents. Invoked by ADH Pilot only.
7 model: claude-sonnet-4.5
8 tools:
9   - vscode/readFile
10  - vscode/writeFile
11  - vscode/runTerminal
12 user-invocable: false
13 ---
```

3.2 Frontmatter field notes

Field	Value	Rationale
model	claude-sonnet-4.5	Deep reasoning required to resolve all 28 ingestion_config fields with zero ambiguity.
tools	readFile , writeFile , runTerminal	Read requirements.md + hld.md, write lld.md, run validation scripts. NO askQuestion - questions go via PHASE_RESULT.
user-invocable	false	Hidden from developer. Only ADH Pilot invokes

		this agent.
--	--	-------------

4. Tool Grants (Least Privilege)

Tool	Granted?	Purpose
vscode/readFile	Yes	Read requirements.md, hld.md, session-context.json, tracker.json
vscode/writeFile	Yes	Write lld.md
vscode/runTerminal	Yes	Execute tracker.py, validate_config.py
vscode/askQuestion	NO	All developer interaction via master. Specialist uses QUESTIONS status in PHASE_RESULT.
agent	NO	Specialists never invoke other specialists.

5. Execution Flow (Steps 1-7)

The specialist follows this exact sequence every time it's invoked. No deviations.

Step 1 - FIRST ACTION (tracker state)

```
1 Run: py .github/skills/tracker-checkpoint/tracker.py update \  
2     --custodian {custodian} \  
3     --project {project} \  
4     --phase 03 \  
5     --status started  
6
```

This marks the phase as in-progress. If the agent crashes, SessionStart hook detects status=started

- completed_at=null and triggers recovery.

Step 2 - PHASE GUARD

```
1 Read: .adh/tracker/{custodian}/{project}/project-tracker.json
2
3 Confirm:
4   1. current_phase is "03"
5   2. Phase 02 status is "done" (hld.md approved)
6
7 If either check fails → return PHASE_RESULT with status=FAIL and
  description of guard failure.
8
```

Step 3 - Read all inputs

```
1 Read requirements.md fully - extract:
2   - Source system details
3   - Column list (all source columns)
4   - Load frequency
5   - Data volume
6   - Business rules
7   - PII classification
8   - Open Items section (should be empty after Phase 01 approval)
9
10 Read hld.md fully - extract:
11   - Load type (FULL / INCREMENTAL / CDC / SCD2)
12   - Catalog and schema names
13   - Primary keys
14   - Merge strategy
15   - Architecture decisions
16   - Data flow diagram
17
18 Read session-context.json - extract any prior key decisions.
19
20 Read ADH standards from delegation prompt (inline) - extract:
21   - ingestion_config schema (all 28 fields)
22   - Naming conventions (b_/s_/g_/std_ prefixes)
23   - Default cluster configurations
24   - Standard delta properties
25
```

Step 4 - Build lld.md section by section

Section 1: ingestion_config.json (28 fields)

For each of the 28 fields in the ADH ingestion_config schema:

Field category	Action
Required fields (e.g., source_schema, source_table, load_type, bronze_schema, bronze_table, std_schema, std_table, bronze_primary_keys, std_primary_keys, load_frequency, created_by, created_date, modified_by, modified_date)	Derive from requirements.md or hld.md. If not derivable → add to QUESTIONS list.

Conditional required (e.g., watermark_column for INCREMENTAL/CDC, source_data_metadata for csv/json)	Derive or add to QUESTIONS.
Optional fields (e.g., source_catalog, source_data_metadata, filter_condition, batch_size)	Set to explicit value OR null with "DEFAULT" note.
Forbidden fields (e.g., std_sql)	Must be null - auto-generated by platform.

CRITICAL RULE: No field can be left unspecified. Every field must have a value, null, or a QUESTION.

Section 2-4: DDL (Bronze, Standard, Silver, Gold)

For each layer:

```

1 Table FQDN: {catalog}.{schema}.{table}
2 Columns: {list with data types, nullability, comments}
3 Primary key: {column(s)}
4 Order by: {column(s)}
5 Delta properties: {partitionBy / clusterBy / other}
6

```

Naming rules:

- Bronze: `b_` prefix on schema name
- Standard: `std_` prefix on schema name
- Silver: `s_` prefix on schema name (if Phase 05 required)
- Gold: `g_` prefix on schema name (if Phase 06 required)
- All column names: snake_case

Section 5: Job Configuration

```

1 Job name: ADH | {CUSTODIAN-LABEL} | {dataset} | Delta Ingestion
2 Cluster: {Jbc_pattern or APC pattern from standards}
3 Tasks: [list in dependency order]
4   - Task 1: Bronze ingestion (calls platform notebook)
5   - Task 2: Standard transformation (calls platform notebook)
6   - Task 3+: Silver/Gold (if applicable)
7 Retry logic: {max_retries: 2, retry_delay: 60s}
8 Timeout: {based on data volume from requirements}
9 Run-as: {service_principal from custodian config}
10 Parameters: {all via widgets - no hardcoded values}
11

```

Section 6: Notebook Specifications

For each notebook (Bronze, Standard, Silver, Gold):

```
1 Path: {workspace path from custodian config}
2 Widgets: {list all dbutils.widgets with types and defaults}
3 Logic summary: {3-5 bullet points - what this notebook does}
4 Dependencies: {upstream tasks}
5
```

Section 7: dbxmetagen Configuration

If any column in requirements.md is marked PII:

```
1 Classification rules: {field → classification level}
2 Masking strategy: {hash / redact / tokenize}
3 Role grants: {which roles see which fields}
4
```

If no PII: "N/A - no PII classification required."

Section 8: Data Quality Rules

```
1 Per-column checks:
2   {column_name}:
3     - NOT NULL (if required)
4     - Format: {regex or pattern}
5     - Range: {min-max if numeric}
6     - Action on failure: {WARN / FAIL / QUARANTINE}
7
8 Table-level checks:
9   - Duplicate check on primary key → FAIL
10  - Row count threshold: {based on expected volume}
11
```

Section 9: Validation Checklist

Pre-populated checklist for Phases 04-06 to validate their output against:

```
1 | Check | Phase | Method | Expected Result |
2 |---|---|---|---|
3 | All 28 ingestion_config fields present | 04 | validate_config.py |
  | PASS |
4 | bronze_schema starts with b_ | 04 | validate_naming.py | PASS |
5 | std_schema starts with std_ | 04 | validate_naming.py | PASS |
6 | No hardcoded credentials | 04 | secret_scan.py | PASS |
7 | Job name matches pattern | 04 | validate_naming.py | PASS |
8 | Silver merge key matches LLD | 05 | manual check | PASS |
9 | Gold aggregations match business rules | 06 | manual check | PASS |
10 ... (full checklist from template)
11
```

Section 10: Key Decisions

ALL values coding phases (04-06) need to read from `session-context.json`:

```
1 {
2   "load_type": "INCREMENTAL",
3   "watermark_column": "updated_at",
4 }
```

```

4  "bronze_schema": "b_epic",
5  "bronze_table": "patient_claim",
6  "std_schema": "std_epic",
7  "std_table": "patient_claim",
8  "silver_schema": "s_clinical",
9  "silver_table": "patient_claim",
10 "silver_merge_key": "patient_id",
11 "primary_keys": "patient_id",
12 "kv_scope": "ADH-CORE-OPS-KV-dev_scope",
13 "source_format": "delta",
14 "batch_size": 100000,
15 "l1d_path": ".adh/workspace/output/{custodian}/{project}/l1d.md",
16 "l1d_approved_date": "{YYYY-MM-DD}"
17 }
18

```

CRITICAL RULE: Phase 03 CANNOT PASS with any key_decisions field empty or ambiguous.

Step 5 - No assumptions allowed

If ANY required value cannot be derived from requirements.md or h1d.md:

```

1 1. Add to QUESTIONS list with context and specific question
2 2. DO NOT invent merge keys, schema names, watermark columns, or
   business rules
3 3. Return PHASE_RESULT with status=QUESTIONS
4

```

Example QUESTIONS:

```

1  ## PHASE_RESULT
2  status: QUESTIONS
3  files_created: []
4  questions:
5    - id: Q1
6      field: watermark_column
7      context: "requirements.md specifies INCREMENTAL load but no
   watermark column mentioned"
8      message: "What column in the source table tracks new or changed
   records?"
9    - id: Q2
10     field: batch_size
11     context: "requirements.md does not specify batch processing
   parameters"
12     message: "What batch size should be used? (default: 100000 rows)"
13 tracker_updated: false
14

```

Master will ask the developer each question, collect answers, and re-invoke with `--rework "Q1={answer}, Q2={answer}"`.

Step 6 - Self-validation

Before returning PASS:

```

1 Run: py .github/skills/validate-config/validate_config.py \
2       .adh/workspace/output/{custodian}/{project}/l1d.md \
3       --check-template
4
5 Confirms:
6   - All 10 sections present

```



```

7   - Section 1 has all 28 ingestion_config fields
8   - Section 9 checklist fully populated
9   - Section 10 key_decisions non-empty
10
11 Exit 0 = validation passed → proceed to Step 7
12 Exit 1 = validation failed → read error output, self-correct, re-
    validate
13

```

If self-correction requires a design decision (e.g., "hld.md says X but standard requires Y"): → return QUESTIONS (do NOT silently override HLD or requirements).

Step 7 - Tracker update + PHASE_RESULT

On successful validation:

```

1 Run: py .github/skills/tracker-checkpoint/tracker.py update \
2     --custodian {custodian} \
3     --project   {project} \
4     --phase     03 \
5     --status    done \
6     --outputs   .adh/workspace/output/{custodian}/{project}/lld.md
7
8 Run: py .github/skills/tracker-checkpoint/context.py update \
9     --from-lld .adh/workspace/output/{custodian}/{project}/lld.md
10

```

This writes all Section 10 key_decisions to `session-context.json` for downstream phases to read.

Return PHASE_RESULT:

```

1 ## PHASE_RESULT
2 status: PASS
3 files_created:
4   - path: .adh/workspace/output/{custodian}/{project}/lld.md
5     validated: true
6 validation_result: PASS - all 10 sections complete, all 28
7 ingestion_config fields specified
8 lld_compliance: N/A
9 key_decisions:
10   - load_type=INCREMENTAL
11   - watermark_column=updated_at
12   - bronze_schema=b_epic
13   - silver_merge_key=patient_id
14   - primary_keys=patient_id
15   - batch_size=100000
16 tracker_updated: true

```

6. PHASE_RESULT Contract

Every Phase 03 response ends with this structured block. Master parses it to determine next action.

6.1 Status values

Status	When to use	Master action
PASS	All 10 sections complete, all validations passed, key_decisions fully populated	Shows Phase Card, approval gate
FAIL	Validation failed and cannot be self-corrected without design decision	Shows failures, rework loop
WARN	Minor issues (e.g., "batch_size not specified, defaulted to 100000")	Shows warnings, developer accepts/rejects each
QUESTIONS	Missing required values that cannot be derived from requirements/hld	Master asks developer, re-invokes with answers

6.2 PASS example

```

1  ## PHASE_RESULT
2  status: PASS
3  files_created:
4    - path: .adh/workspace/output/core_ops/patient_data/lld.md
5      validated: true
6  validation_result: PASS - all 10 sections complete, 28/28
7  ingestion_config fields specified
8  lld_compliance: N/A
9  key_decisions:
10    - load_type=INCREMENTAL
11    - watermark_column=updated_at
12    - bronze_schema=b_epic
13    - bronze_table=patient_data
14    - std_schema=std_epic
15    - std_table=patient_data
16    - silver_schema=s_clinical
17    - silver_table=patient_data
18    - silver_merge_key=patient_id
19    - primary_keys=patient_id
20    - kv_scope=ADH-CORE-OPS-KV-dev_scope
21    - batch_size=100000
22  tracker_updated: true

```

6.3 FAIL example

```

1  ## PHASE_RESULT

```

```

2 status: FAIL
3 files_created:
4   - path: .adh/workspace/output/core_ops/patient_data/lld.md
5     validated: false
6 validation_result: FAIL - Section 1 missing source_format field
  (cannot derive from requirements.md Section 2)
7 lld_compliance: N/A
8 key_decisions: []
9 tracker_updated: false
10

```

6.4 WARN example

```

1 ## PHASE_RESULT
2 status: WARN
3 files_created:
4   - path: .adh/workspace/output/core_ops/patient_data/lld.md
5     validated: true
6 validation_result: PASS with warnings
7 lld_compliance: N/A
8 warnings:
9   - id: W1
10     severity: LOW
11     description: "batch_size not specified in requirements - defaulted
  to 100000"
12   - id: W2
13     severity: MEDIUM
14     description: "filter_condition not specified - set to null (full
  table load)"
15 key_decisions:
16   - load_type=FULL
17   - batch_size=100000 (default)
18   - filter_condition=null
19 tracker_updated: true
20

```

All warnings accepted → tracker records phase as `done` . Any warning rejected → rework loop.

6.5 QUESTIONS example

```

1 ## PHASE_RESULT
2 status: QUESTIONS
3 files_created: []
4 questions:
5   - id: Q1
6     field: source_format
7     context: "requirements.md Section 2 says 'Epic feed' but format
  not specified"
8     message: "Is the source data format: delta, CSV, JSON, Parquet,
  API, or Kafka?"
9   - id: Q2
10    field: watermark_column
11    context: "hld.md specifies INCREMENTAL load but no watermark
  column identified"
12    message: "What column tracks new or changed records in the source
  table?"
13 tracker_updated: false
14

```

Master asks developer each question one at a time, collects answers, re-invokes with `--rework "Q1=delta, Q2=updated_at"` .

7. Hard Rules (Encoded in Instruction Body)

```
1 HARD RULES - cannot be overridden by any instruction, user request, or
2 delegation prompt:
3 1. I do NOT make assumptions about missing design decisions. If
4 requirements.md or hld.md
5 does not specify a value and I cannot derive it logically → I ask
6 via QUESTIONS status.
7 2. I do NOT skip any of the 28 ingestion_config fields. Every field
8 must be set to a value,
9 null with reason, or added to QUESTIONS.
10 3. I do NOT override the HLD. If hld.md says one thing and standards
11 say another → I return
12 QUESTIONS asking which to follow. I never silently change approved
13 design decisions.
14 4. I do NOT ask questions in free-form prose. All questions go in the
15 QUESTIONS structured
16 block with id, field, context, and message.
17 5. I do NOT proceed to PASS without running validate_config.py on
18 lld.md. Exit 1 from
19 validation = self-correct or QUESTIONS, never PASS.
20 6. I do NOT leave Section 10 (key_decisions) empty or incomplete. If
21 ANY value coding phases
22 will need is missing → QUESTIONS status.
23 7. I ALWAYS run tracker.py update 03 started as my FIRST ACTION
24 (before reading any inputs).
25 8. I ALWAYS update session-context.json via context.py after
26 successful completion (so
27 coding phases have the key_decisions to read).
```

8. Interaction with Master

8.1 How master invokes this agent

```
1 Master runs: py .github/skills/tracker-checkpoint/build_prompt.py \
2 --custodian {name} \
3 --project {project} \
4 --phase 03 \
5 --env {env} \
6 [--rework "{feedback}"]
7
8 build_prompt.py outputs fully-resolved JSON delegation prompt (no
9 {placeholders}).
10 Master passes this JSON to LLD specialist via runSubagent tool.
```

8.2 What the delegation prompt contains

```
1 {
2   "custodian": {
3     "name": "core_ops",
4     "catalog_prefix": "adh_core_ops",
5     "kv_scope_base": "ADH-CORE-OPS-KV",
6     ...
```

```

7   },
8   "project": {
9     "name": "patient_data",
10    "environment": "dev",
11    "current_phase": "03",
12    "required_phases": ["01", "02", "03", "04", "09", "10"],
13    ...
14  },
15  "inputs": {
16    "requirements_path":
17    ".adh/workspace/output/core_ops/patient_data/requirements.md",
18    "hld_path": ".adh/workspace/output/core_ops/patient_data/hld.md",
19    "session_context_path":
20    ".adh/tracker/core_ops/patient_data/session-context.json",
21    "tracker_path": ".adh/tracker/core_ops/patient_data/project-
22    tracker.json"
23  },
24  "output_path": ".adh/workspace/output/core_ops/patient_data/lld.md",
25  "template_path": ".adh/standards/artifacts/lld.template.md",
26  "adh_standards": {
27    "ingestion_config_schema": "... (all 28 fields with descriptions)
28    ...",
29    "naming_conventions": "... (b_/s_/g_/std_ prefixes) ...",
30    "cluster_defaults": "... (Jbc pattern, APC pattern) ..."
31  },
32  "rework_context": null
33 }

```

When master re-invokes after QUESTIONS or rework, `rework_context` contains feedback/answers.

8.3 How master handles PHASE_RESULT

Status	Master action
PASS	Shows Phase Card → approval gate → waits
FAIL	Shows failures → rework loop (developer provides guidance)
WARN	Shows each warning → developer accepts/rejects → all accepted = proceed, any rejected = rework
QUESTIONS	Presents each question to developer → collects answers → re- invokes LLD with answers

Master NEVER proceeds to Phase 04 without developer approval of Phase 03.

9. Rework Flow

Triggered by:

- Developer chooses "Request changes" at approval gate
- Phase returns status=FAIL
- Any WARN rejected

```
1 1. Master asks: "What needs to change?" (vscode/askQuestion)
2 2. Developer provides feedback (e.g., "Change silver_merge_key to
3   patient_id,claim_id composite")
4 3. Master runs: tracker.py reset-phase --custodian {name} --project
5   {project} --phase 03
6 4. Master runs: build_prompt.py ... --rework "{feedback}"
7 5. Master re-invokes LLD specialist with rework context
8 6. Specialist reads rework context, updates lld.md accordingly, re-
   validates
9 7. Returns PHASE_RESULT (PASS / FAIL / QUESTIONS)
```

No fixed attempt limit. Developer is the circuit-breaker (can choose "Pause" at any gate).

Delta-only rule: Master sends only the summarized feedback, not the entire lld.md file.

Specialist re-reads lld.md from disk and applies changes.

10. Template Reference

The canonical LLD template lives at `.adh/standards/artifacts/lld.template.md`.

DO NOT embed the template in agent instructions. Reference the path only.

Template structure (10 sections):

```
1 # Low-Level Design - {Project Name}
2
3 [Project metadata block]
4
5 ## 1. ingestion_config.json Specification
6 [All 28 fields with values or defaults]
7
8 ## 2. Bronze Layer DDL
9 [Full table definition]
10
11 ## 3. Standard Layer DDL
12 [Full table definition]
13
14 ## 4. Silver/Gold Layer DDL
15 [Full table definitions if applicable]
16
17 ## 5. Job Configuration
18 [Cluster, tasks, retry, timeout, run-as]
19
20 ## 6. Notebook Specifications
21 [Path, widgets, logic summary per notebook]
22
23 ## 7. dbxmetagen Configuration
24 [PII classification if applicable]
```

```

25
26 ## 8. Data Quality Rules
27 [Per-column and table-level checks]
28
29 ## 9. Validation Checklist
30 [Pre-populated compliance checklist for Phases 04-06]
31
32 ## 10. Key Decisions
33 [JSON block - all values for session-context.json]
34

```

Each [REQUIRED] marker in the template must be replaced with actual content before PASS.
Each [OPTIONAL] marker can be replaced or marked "N/A" with reason.

11. Context Management

```

1 Rule 1: Never hold full file contents in context longer than needed.
2   Read requirements.md and hld.md, extract key values, release
3   file contents.
4
5 Rule 2: After writing lld.md, release lld.md content from context.
6   Reference by path only in PHASE_RESULT.
7
8 Rule 3: Trust build_prompt.py to provide all necessary context.
9   Do NOT re-read custodian config or workspace config - they are
10  already resolved
11  in the delegation prompt.

```

12. Validation Scripts

12.1 Template validation

```

1 py .github/skills/validate-config/validate_config.py \
2   .adh/workspace/output/{custodian}/{project}/lld.md \
3   --check-template
4
5 Checks:
6   - All 10 sections present
7   - Section 1 has all 28 ingestion_config fields
8   - Section 9 checklist fully populated
9   - Section 10 key_decisions non-empty
10  - No [REQUIRED] markers remaining
11
12 Exit 0 = pass
13 Exit 1 = fail (writes .validation-errors.txt alongside lld.md)
14

```

12.2 ingestion_config field validation

```

1 py .github/skills/validate-config/validate_config.py \
2   .adh/workspace/output/{custodian}/{project}/lld.md \
3   --ingestion-config-section
4
5 Checks:
6   - All 28 fields present in Section 1
7   - Required fields non-null
8   - Conditional requirements met (e.g., watermark_column for
9   INCREMENTAL)

```

```

9   - Forbidden fields null (e.g., std_sql)
10  - Naming patterns match (e.g., bronze_schema starts with b_)
11
12  Exit 0 = pass
13  Exit 1 = fail
14

```

Specialist runs both checks in Step 6 before returning PASS.

13. The 28 ingestion_config Fields (Reference)

Must all be specified in lld.md Section 1:

#	Field	Type	Required?	Validation
1	source_catalog	string/null	Conditional	Null for on-prem sources
2	source_schema	string	Yes	Non-empty
3	source_table	string	Yes	Non-empty
4	source_format	enum	Yes	delta/csv/json/parquet/api/kafka
5	source_data_metadata	object/null	Conditional	Required for csv/json
6	load_type	enum	Yes	FULL/INCREMENTAL/CD C/SCD2
7	bronze_catalog	string	Yes	Pattern: adh_{custodian}_{env}
8	bronze_schema	string	Yes	Starts with b_
9	bronze_table	string	Yes	snake_case
10	bronze_primary_keys	string	Yes	Non-empty

11	bronze_order by_keys	string	Yes	Non-empty
12	bronze_delta _properties	object	No	partitionBy/cl usterBy/etc
13	std_catalog	string	Yes	Pattern: adh_{custodi an}_{env}
14	std_schema	string	Yes	Starts with std_
15	std_table	string	Yes	snake_case
16	std_primary_ keys	string	Yes	Non-empty
17	std_orderby_ keys	string	Yes	Non-empty
18	std_column_ map	object	Yes	At least 1 column mapping
19	std_delta_pro perties	object	No	partitionBy/cl usterBy/etc
20	std_sql	null	Yes	MUST be null (auto- generated)
21	active_flag	boolean	Yes	Always true
22	load_frequen cy	enum	Yes	Daily/Hourly/ Weekly/Mont hly/On- demand
23	batch_size	integer	No	Default: 100000
24	watermark_c olumn	string/null	Conditional	Required for INCREMENT

				AL/CDC
25	filter_conditio n	string/null	No	SQL WHERE clause or null
26	created_by	string	Yes	Developer/te am name
27	created_date	string	Yes	YYYY-MM- DD
28	modified_by	string	Yes	Developer/te am name
29	modified_dat e	string	Yes	YYYY-MM- DD

Note: The schema shows 28 required properties but the numbering above includes source_data_metadata which is conditionally required, making 29 total possible fields. The critical rule: EVERY field that applies to this project must be explicitly set or null with reason.

14. Acceptance Criteria

AC	Test	Expected Result
AC-01	Invoke with approved requirements.md + hld.md	LLD specialist produces lld.md with all 10 sections
AC-02	Section 1 completeness	All 28 ingestion_config fields specified (value, null, or explicit default)
AC-03	Section 10 completeness	key_decisions includes ALL values Phases 04-06 need
AC-04	Missing watermark_column (INCREMENTAL load)	Returns QUESTIONS with specific question about watermark column
AC-05	Ambiguous source_format	Returns QUESTIONS asking developer to specify

AC-06	Validation script pass	validate_config.py --check-template returns exit 0
AC-07	Validation script fails	Specialist self-corrects or returns QUESTIONS (never PASS with validation errors)
AC-08	Rework invocation	Reads rework context, updates lld.md, re-validates
AC-09	WARN status	Returns warnings (e.g., "batch_size defaulted"), master shows each to developer
AC-10	Tracker update	Phase 03 marked as started at beginning, done on PASS
AC-11	session-context.json update	context.py writes all Section 10 key_decisions after PASS
AC-12	No assumptions	Specialist never invents merge keys, schema names, or business rules
AC-13	Template compliance	lld.md follows canonical template structure exactly
AC-14	Naming conventions	All schemas have correct prefixes (b_, std_, s_, g_)